

Universidad de las Fuerzas Armadas – ESPE

Departamento de Ciencias de la Computación

Ingeniería en Tecnologías de la Información

Aplicaciones Distribuidas

Práctica de laboratorio 2:

Sistema de Votación en Tiempo Real mediante WebSockets y Broadcast

Autor:

Franco Antonio Pillajo Peralta

José Sebastían Lojan Vicuña

Docente:

Ing. Darwin Rene Arias Martinez

Quito – Ecuador

1. Objetivos

1.1 Objetivo general

Implementar y evaluar un sistema de votación web en tiempo real mediante React, FastAPI, WebSocket y MySQL, capaz de almacenar votos y difundir automáticamente los resultados a múltiples clientes conectados.

1.2 Objetivos específicos

- Diseñar una arquitectura de tres capas que separe la interfaz web, el procesamiento de votaciones y el almacenamiento persistente.
- Construir un gestor de conexiones WebSocket que registre clientes activos y publique el resultado de cada voto mediante Broadcast.
- Implementar el incremento transaccional de contadores en MySQL y representar resultados y porcentajes con una interfaz responsive desarrollada en React.
- Comprobar el comportamiento del sistema mediante consultas de persistencia, tres navegadores sincronizados, una ejecución concurrente de diez votos y la detección de desconexión del servidor.

Alcance. La práctica corresponde a una simulación académica con tres partidos ficticios. Las pruebas se ejecutaron en un entorno local, con una única instancia de FastAPI. No se implementaron controles de identificación del votante ni despliegue de múltiples servidores.

2. Marco teórico

2.1 Aplicaciones distribuidas y cliente-servidor

Una aplicación distribuida coordina componentes que se comunican mediante una red. La arquitectura cliente-servidor separa presentación, procesamiento y persistencia. En esta práctica, los navegadores son clientes independientes y FastAPI coordina la votación.

2.2 WebSocket y comunicación bidireccional

WebSocket establece un canal persistente que permite intercambiar mensajes en ambas direcciones, sin abrir una nueva petición HTTP para cada actualización. El protocolo se define en el RFC 6455 (Fette & Melnikov, 2011). En la práctica, cada navegador mantiene una conexión con el endpoint /ws de FastAPI.

2.3 Patrón Broadcast

Broadcast es una estrategia de difusión en la que un emisor produce un mensaje que se envía a todos los receptores de un conjunto. En el proyecto, ConnectionManager mantiene los clientes conectados y, tras confirmarse un voto, distribuye el mismo estado actualizado a todos ellos (FastAPI, s. f.).

2.4 React, JSON y persistencia relacional

React actualiza la interfaz al cambiar su estado y useEffect administra la conexión externa (React, s. f.). Los mensajes viajan en JSON, pero MySQL mantiene los votos en una tabla relacional; JSON no requiere una base de datos documental.

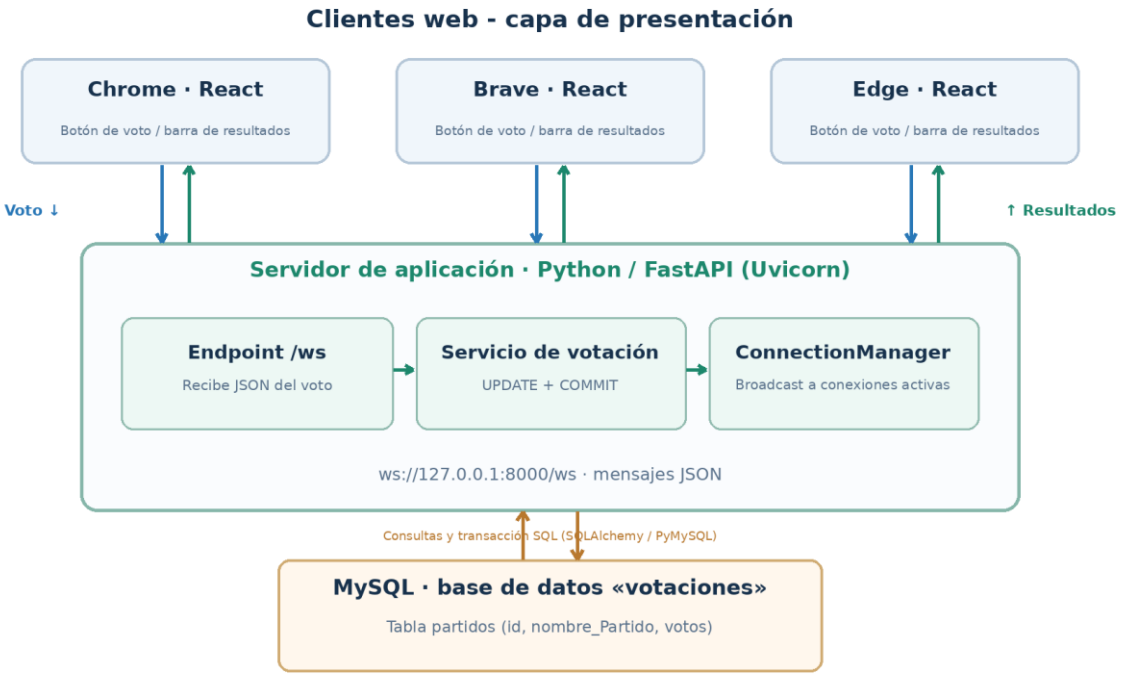
3. Arquitectura del sistema

3.1 Diagrama lógico de arquitectura

La Figura 1 representa tres navegadores conectados a una única instancia de FastAPI. Los clientes envían votos por WebSocket; la capa de aplicación actualiza MySQL y difunde los resultados nuevamente a todos los clientes. El diagrama corresponde al laboratorio realizado y no presupone servidores adicionales.

Figura 1

Arquitectura cliente-servidor con difusión Broadcast y persistencia MySQL



Nota. Diagrama de elaboración propia (2026). Las flechas azules representan el envío de votos; las verdes, las actualizaciones difundidas; y las naranjas, las operaciones SQL.

3.2 Responsabilidades y flujo de operación

La capa de presentación utiliza React y HTML5 semántico para mostrar tres candidatos ficticios, barras de progreso y botones de votación. La capa de aplicación recibe mensajes JSON

en FastAPI, comprueba el partido solicitado, actualiza el contador y coordina la distribución mediante ConnectionManager. La capa de persistencia es la base MySQL votaciones, con una única tabla partidos (id, nombre_Partido y votos).

El recorrido de cada voto sigue la secuencia: cliente React → WebSocket /ws → servicio de votación → transacción MySQL → consulta de resultados → Broadcast → actualización del estado de React. Al conectarse un cliente nuevo, el servidor también le entrega el estado inicial almacenado.

3.3 Pila tecnológica

La Tabla 1 resume las tecnologías observadas en el desarrollo. Se consignan versiones identificables en las capturas y se distinguen de las que requieren una consulta directa al entorno instalado, evitando atribuir versiones no comprobadas.

Tabla 1

Tecnologías y versiones documentadas

Componente	Aplicación y versión
Python	Lenguaje del backend; versión exacta no registrada en la evidencia.
FastAPI	Rutas HTTP y WebSocket; metadatos de descarga: 0.161.1.
Uvicorn	Servidor ASGI; metadatos de descarga: 0.54.0.
SQLAlchemy	Acceso SQL y transacciones; metadatos de descarga: 2.1.6.
PyMySQL	Controlador de conexión MySQL; metadatos de descarga: 1.2.3.
python-dotenv	Lectura de configuración .env; metadatos de descarga: 1.2.4.
React / JavaScript	Interfaz dinámica; versión exacta de React no documentada.
Vite	Herramienta de desarrollo frontend; versión 8.3.5 observada.
MySQL / Workbench	Almacenamiento y administración; versión no documentada.
WebSocket / JSON	Protocolo bidireccional y formato de intercambio; no son versiones de paquetes.

4. Desarrollo e implementación

4.1 Creación de la base de datos MySQL

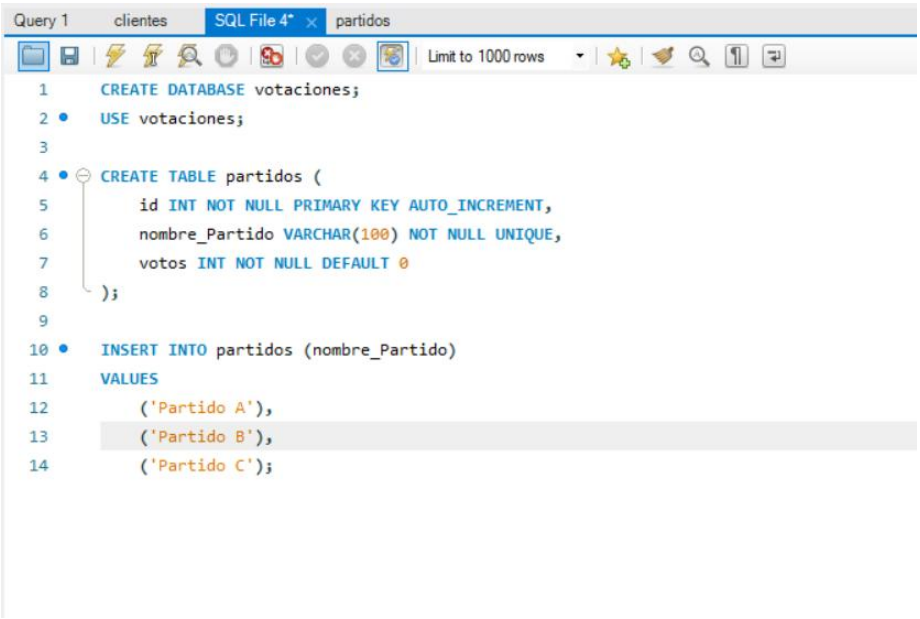
Se creó la base votaciones y la tabla partidos mediante MySQL Workbench. El campo id es una clave primaria autoincremental; nombre_Partido identifica de forma única cada opción; y votos mantiene un contador entero con valor inicial cero. Se insertaron Partido A, Partido B y Partido C.

```
CREATE TABLE partidos (
  id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
  nombre_Partido VARCHAR(100) NOT NULL UNIQUE,
  votos INT NOT NULL DEFAULT 0
);
```

Como se observa en la Figura 2, el esquema y la carga inicial se ejecutaron en MySQL Workbench.

Figura 2

Definición SQL de la base votaciones y la tabla partidos



Nota. Captura propia del laboratorio (2026). Se observan la definición de los campos y la inserción de los tres partidos.

4.2 Organización del backend y conexión a MySQL

El backend se estructuró en cuatro módulos: main.py contiene los endpoints; database.py construye el engine de SQLAlchemy; connection_manager.py administra las sesiones WebSocket; y votacion_service.py contiene la consulta de resultados y la actualización de contadores. Se creó un entorno virtual de Python para aislar dependencias y se configuró la conexión con variables de entorno en un archivo .env, excluido de Git mediante .gitignore.

La organización del backend se documentó desde el explorador de Visual Studio Code. También se configuró .gitignore para excluir el entorno virtual y el archivo local de credenciales al utilizar un repositorio de código.

Figura 3

Estructura del backend y archivo de exclusiones de Git

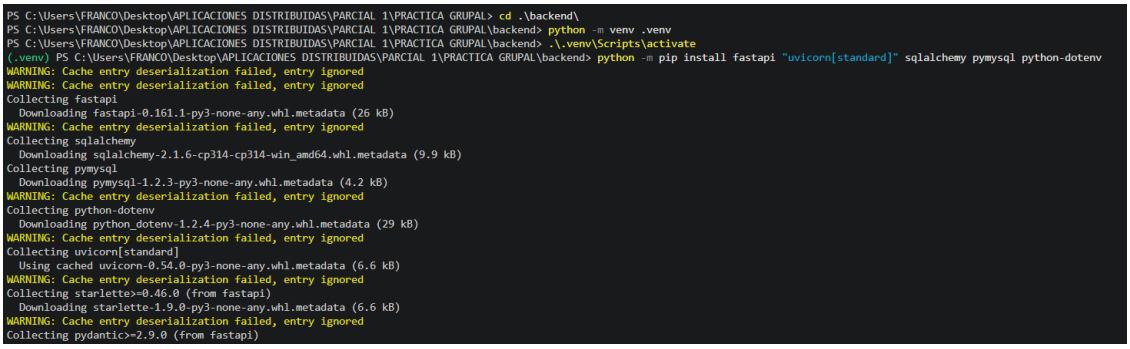


Nota. Composición de dos capturas propias (2026): (a) módulos Python, entorno virtual y configuración local; (b) exclusión de .venv, .env y __pycache__.

Las dependencias se instalaron dentro de un entorno virtual de Python, evitando mezclarlas con paquetes globales del sistema.

Figura 4

Instalación de dependencias del entorno virtual



Nota. Captura propia de PowerShell (2026). Se registra la instalación de paquetes necesarios para FastAPI y el acceso a MySQL.

El módulo database.py lee variables de entorno con python-dotenv, construye la URL mysql+pymysql y crea el engine de SQLAlchemy para las operaciones SQL.

Figura 5

Configuración de la conexión a MySQL en database.py



Nota. Captura propia del código fuente (2026). Las credenciales se recuperan desde variables de entorno, no están escritas literalmente en el módulo.

La conexión utiliza el dialecto mysql+pymysql y pool_pre_ping=True para comprobar conexiones reutilizadas. Las credenciales no se incluyen en este informe ni en sus capturas por seguridad.

4.3 Implementación de WebSocket y ConnectionManager

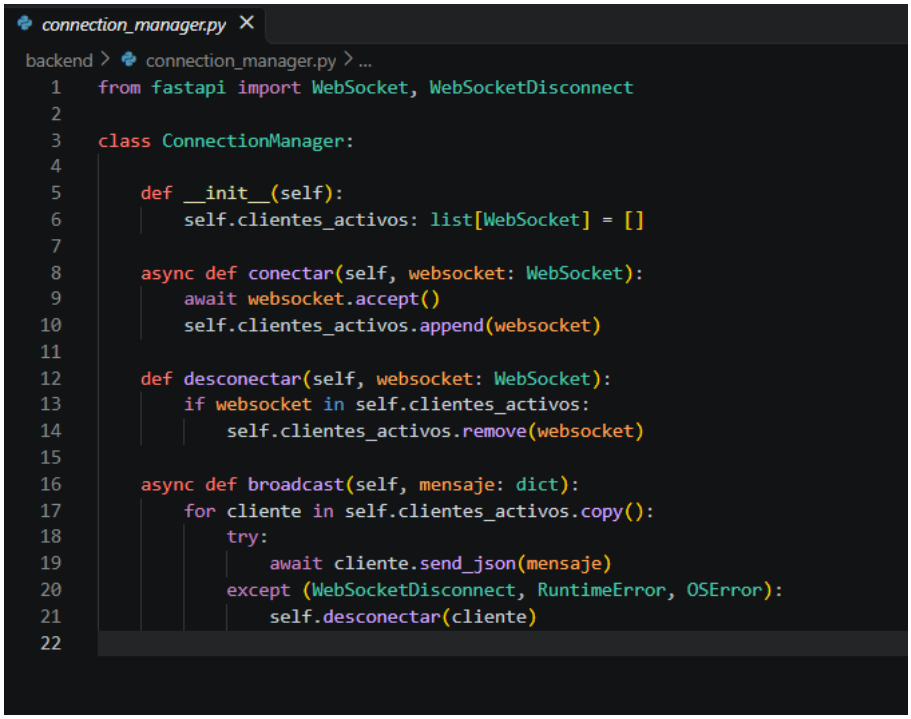
FastAPI expone GET /partidos para consultar los datos por HTTP y /ws para establecer comunicación WebSocket. El gestor de conexiones acepta clientes, los conserva en una lista

activa y los retira cuando termina la conexión. El método broadcast() recorre las conexiones y envía el mismo diccionario de resultados como JSON.

La Figura 6 muestra los métodos conectar(), desconectar() y broadcast() implementados en connection_manager.py. Esta gestión en memoria resulta adecuada para la demostración con un solo proceso servidor.

Figura 6

Gestor de conexiones WebSocket y método Broadcast

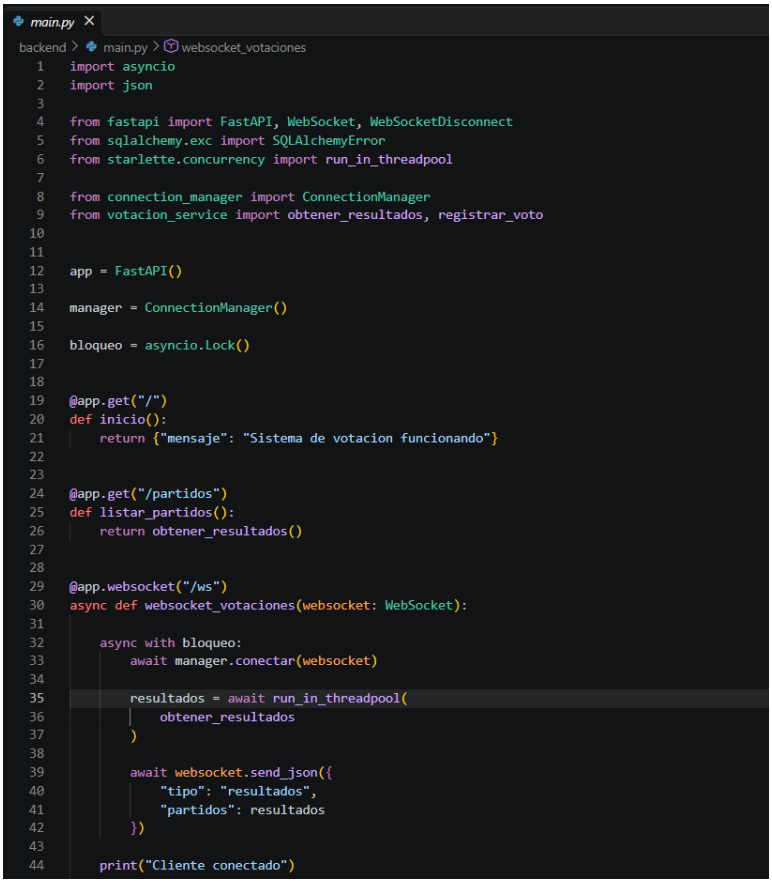


Nota. Captura propia del archivo connection_manager.py (2026). La lista de clientes activos permite difundir los resultados a todos los navegadores conectados.

El archivo main.py concentra los endpoints, la aceptación de nuevas sesiones y la recepción de mensajes JSON. Su implementación se documenta en las tres capturas siguientes.

Figura 7

Rutas HTTP, endpoint WebSocket y aceptación de clientes



Nota. Captura propia de main.py (2026). Se muestra la inicialización de FastAPI, el gestor y el envío inicial de resultados al conectar.

Figura 8

Recepción de mensajes y coordinación de votos en main.py



```
main.py
backend > main.py > websocket_votaciones
30  async def websocket_votaciones(websocket: WebSocket):
46      try:
47          while True:
48
49              try:
50                  mensaje = await websocket.receive_json()
51              except json.JSONDecodeError:
52                  await websocket.send_json({
53                      "tipo": "error",
54                      "mensaje": "JSON invalido"
55                  })
56                  continue
57
58              if not isinstance(mensaje, dict):
59                  await websocket.send_json({
60                      "tipo": "error",
61                      "mensaje": "Formato de mensaje invalido"
62                  })
63                  continue
64
65              if mensaje.get("tipo") != "votar":
66                  await websocket.send_json({
67                      "tipo": "error",
68                      "mensaje": "Accion no reconocida"
69                  })
70                  continue
71
72              partido_id = mensaje.get("partido_id")
73
74              if type(partido_id) is not int:
75                  await websocket.send_json({
76                      "tipo": "error",
77                      "mensaje": "ID de partido invalido"
78                  })
79                  continue
80
81              async with bloqueo:
82
83                  try:
84                      resultados = await run_in_threadpool(
85                          registrar_voto,
86                          partido_id
87                      )
88
89                  except ValueError as error:
90                      await websocket.send_json({
```

Nota. Captura propia de main.py (2026). El ciclo de recepción valida el mensaje y ejecuta el servicio de votación mediante un hilo de trabajo.

Figura 9

Difusión Broadcast y gestión de desconexiones en main.py



Nota. Captura propia de main.py (2026). Se observa manager.broadcast y la eliminación de sesiones desconectadas.

4.4 Registro de votos y consistencia

El servicio de votación utiliza SQL parametrizado, ejecuta el incremento dentro de engine.begin() y solo después consulta los totales. La instrucción UPDATE evita leer el contador en Python, incrementarlo y escribir un valor calculado (Oracle, s. f.). SQLAlchemy confirma la transacción al finalizar correctamente el bloque (SQLAlchemy, s. f.). Un bloqueo asíncrono coordina el orden de las actualizaciones y difusiones en el proceso FastAPI.

```
UPDATE partidos
SET votos = votos + 1
WHERE id = :partido_id;
```

La Figura 10 documenta el servicio encargado de persistir los votos. La operación se confirma antes del Broadcast, de modo que los clientes reciben un resultado ya guardado en MySQL.

Figura 10

Servicio de actualización transaccional de los votos

votacion_service.py

backend > votacion_service.py > ...

```python
def obtener_resultados():
    with engine.connect() as conexion:
        consulta = text(
            "SELECT id, nombre_Partido, votos "
            "FROM partidos ORDER BY id"
        )

        resultado = conexion.execute(consulta)

        return [
            dict(fila)
            for fila in resultado.mappings().all()
        ]


def registrar_voto(partido_id: int):

    with engine.begin() as conexion:
        consulta = text(
            "UPDATE partidos "
            "SET votos = votos + 1 "
            "WHERE id = :partido_id"
        )

        resultado = conexion.execute(
            consulta,
            {"partido_id": partido_id}
        )

        if resultado.rowcount != 1:
            raise ValueError("El partido no existe")

    return obtener_resultados()

```

Nota. Captura propia del archivo votacion_service.py (2026). Se aprecian el UPDATE parametrizado, la transacción y la consulta posterior.

4.5 Interfaz React, HTML5 y diseño responsive

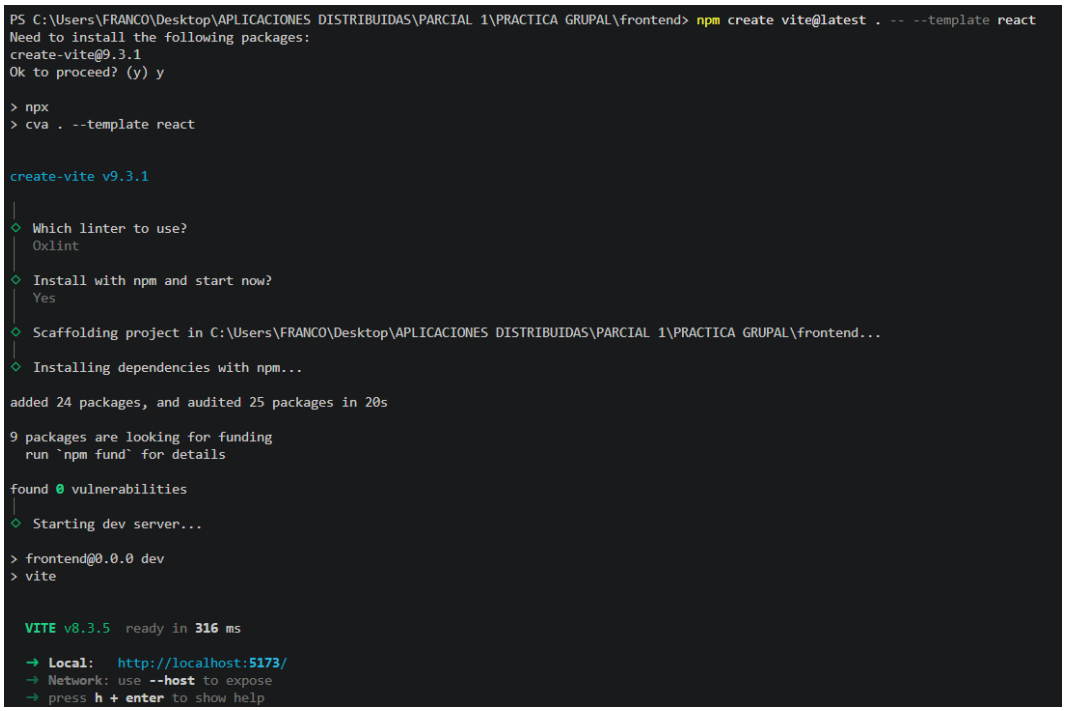
El proyecto frontend se creó con Vite y React (Vite, s. f.). En App.jsx, useEffect establece la conexión WebSocket y procesa mensajes de tipo resultados; useState almacena los partidos y activa el renderizado de nuevas barras y contadores. Al pulsar Votar, el cliente envía

{"tipo":"votar","partido_id":N} sin modificar localmente el conteo, puesto que la fuente de verdad es MySQL.

La aplicación React se inicializó con la plantilla de Vite; después se verificó que el proyecto arrancara correctamente antes de integrar WebSocket y personalizar su presentación.

Figura 11

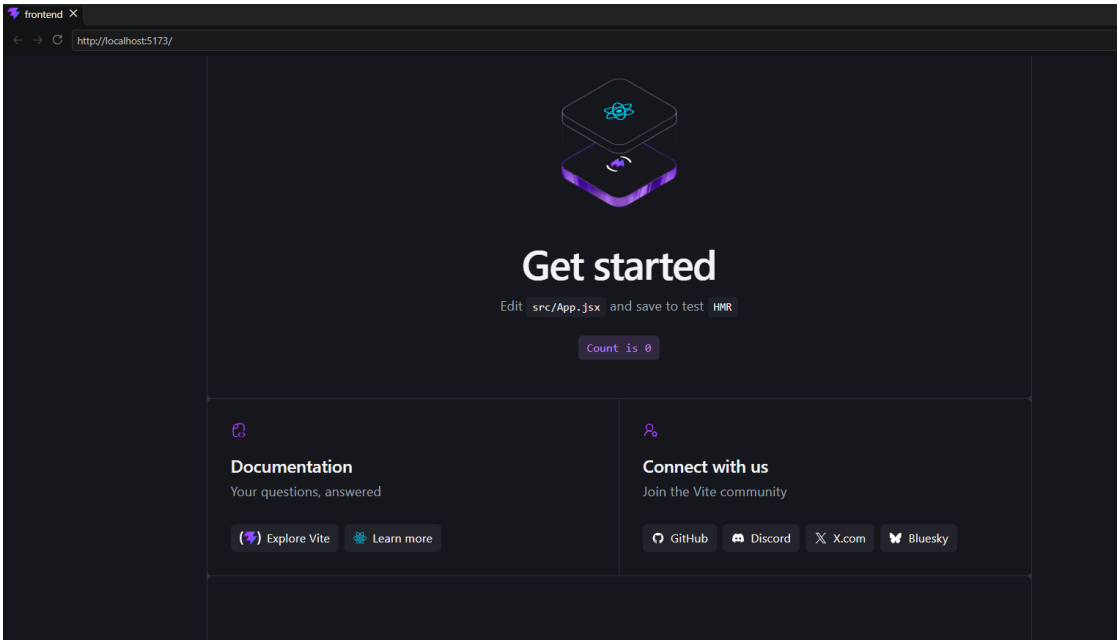
Creación del proyecto React mediante Vite



Nota. Captura propia de PowerShell (2026). Se aprecia el comando de inicialización y la preparación del servidor de desarrollo.

Figura 12

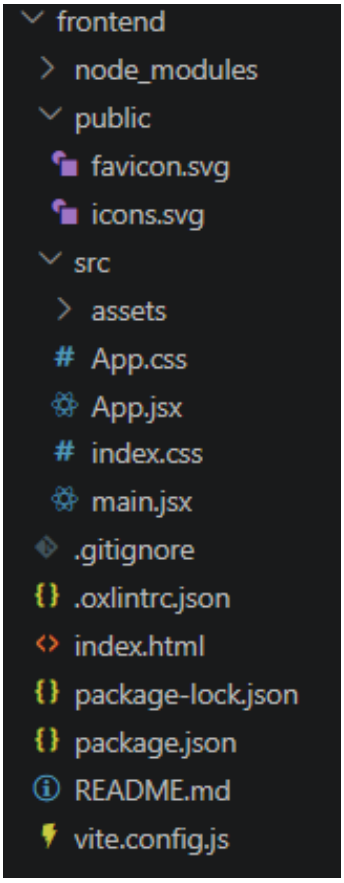
Pantalla inicial de React y Vite antes del desarrollo



Nota. Captura propia del navegador (2026). Verificación preliminar de que el entorno de frontend funciona antes del reemplazo por la interfaz electoral.

Figura 13

Directorio del frontend generado con Vite



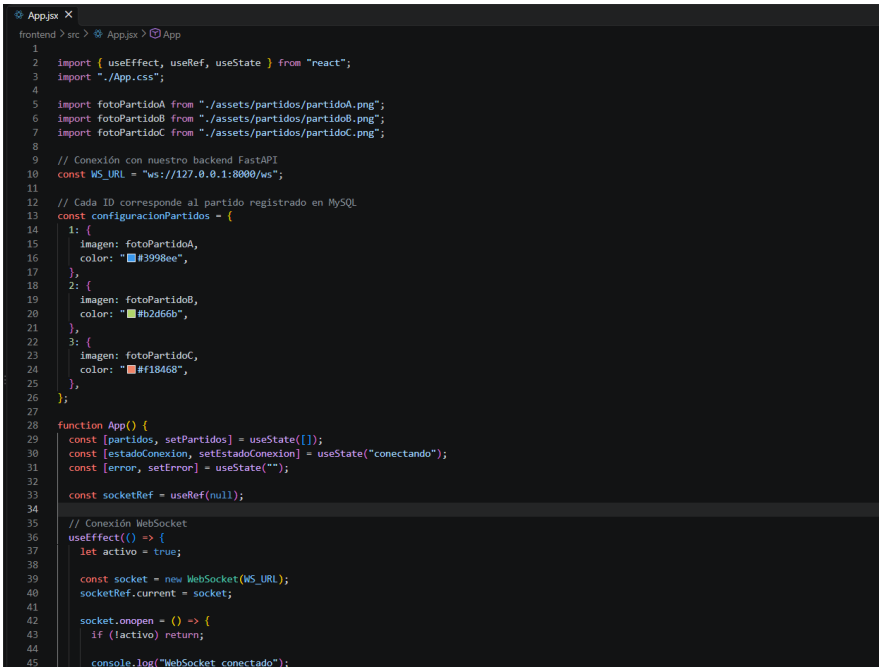
Nota. Captura propia de Visual Studio Code (2026). Se identifican src, assets, App.jsx, App.css, index.css y package.json.

Se diseñaron tres tarjetas horizontales con fotografías locales, nombre del partido, botón, barra de progreso y porcentaje. El diseño usa HTML5 semántico y hojas CSS con Grid, Flexbox y media queries. En resoluciones menores se reorganizan las columnas para conservar legibilidad y accesibilidad. Vite se ejecutó en localhost:5173 y FastAPI en 127.0.0.1:8000.

La Figura 14 muestra la configuración de imágenes y el inicio de la conexión WebSocket en App.jsx. Las capturas de la interfaz ejecutada se incluyen más adelante en las pruebas con tres navegadores.

Figura 14

Componente React con fotografías locales y conexión WebSocket

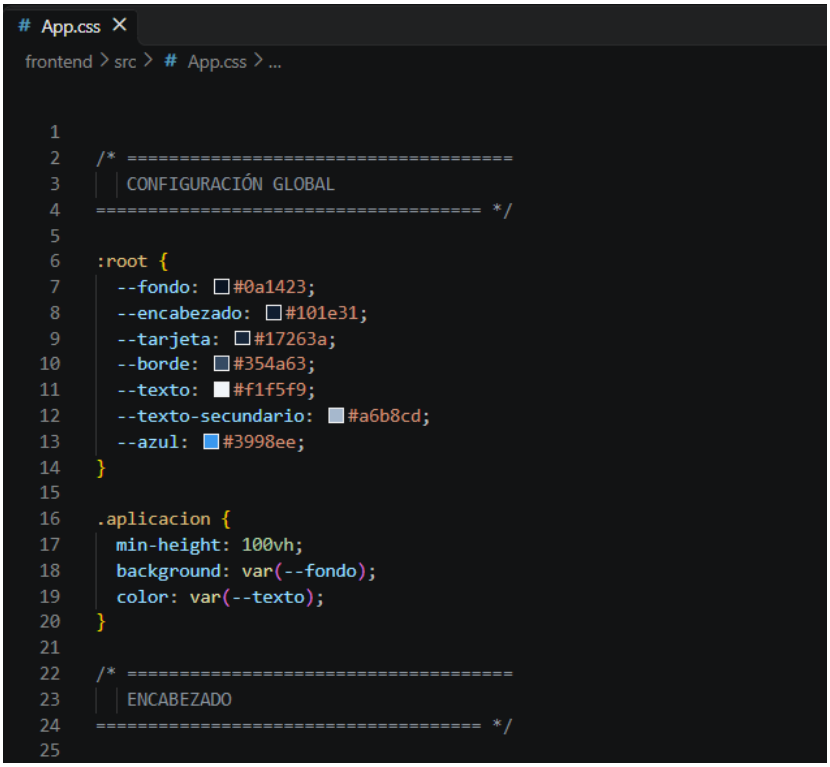


Nota. Captura propia de App.jsx (2026). Se aprecian los imports de React, la URL /ws, la asignación de imágenes y los estados del componente.

La interfaz se diseñó con estilos propios en App.css e index.css. Las reglas visuales definen el tema oscuro, las barras de progreso, las tarjetas de partidos y los ajustes adaptativos para distintos anchos de pantalla.

Figura 15

Definición de estilos de la interfaz en App.css



Nota. Captura propia de App.css (2026). Se aprecian los colores base y la configuración general de la aplicación.

Figura 16

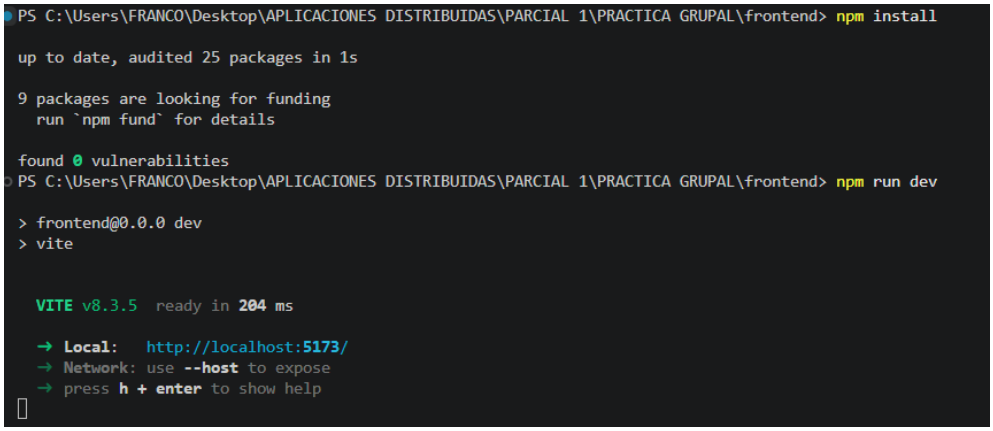
Estilos globales del documento en index.css



Nota. Captura propia de index.css (2026). Configuración de estilos base, tipografía y comportamiento de la página.

Figura 17

Servidor Vite disponible en el puerto 5173



Nota. Captura propia de PowerShell (2026). La ejecución de npm run dev publica la aplicación React en localhost:5173.

5. Pruebas y resultados

Se realizaron pruebas funcionales en el entorno local con los tres componentes activos. Las evidencias provienen del archivo de capturas adjuntado y se organizaron según el requisito que permiten verificar. No se reportan métricas de latencia ni carga sostenida porque no fueron medidas.

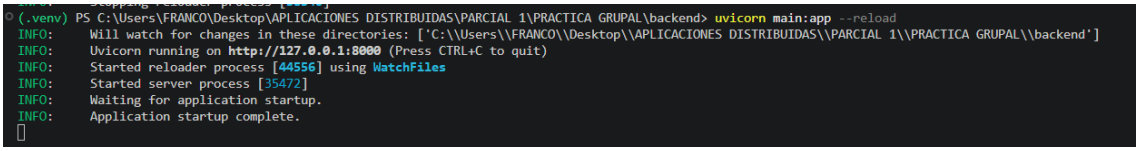
5.1 Consulta HTTP y recepción por WebSocket

Se comprobó GET /partidos desde la documentación interactiva de FastAPI, con respuesta HTTP correcta, y se abrió una conexión WebSocket desde la consola del navegador. La recepción de mensajes de tipo resultados confirmó que el servidor transmitía una lista con los tres partidos. Las Figuras 6 y 7 documentan ambas etapas.

La primera comprobación técnica verificó que el servidor Uvicorn estuviera activo y atendiera peticiones en el puerto 8000, condición previa al intercambio por WebSocket.

Figura 18

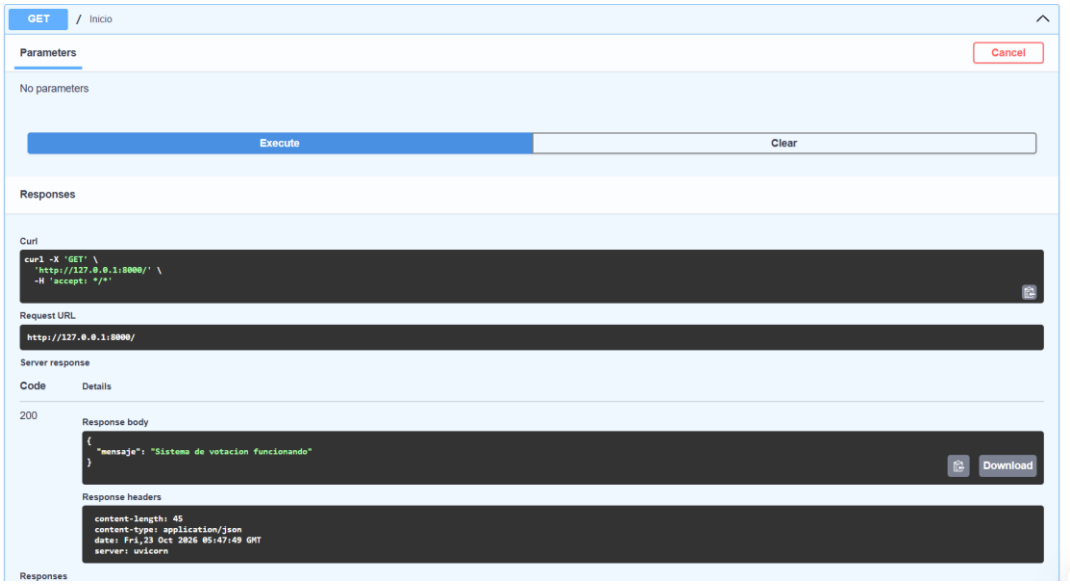
Arranque y estado operativo del servidor Uvicorn



Nota. Captura propia del terminal (2026). Se observa que la aplicación de FastAPI completa el inicio del servidor.

Figura 19

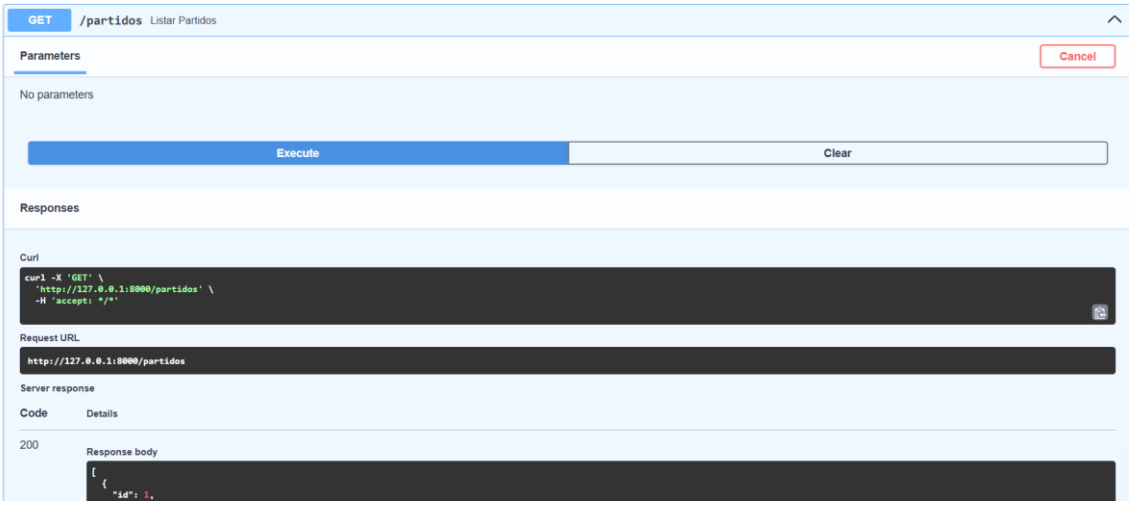
Consulta HTTP desde la documentación interactiva de FastAPI



Nota. Captura propia (2026). Se observan los endpoints HTTP y la respuesta obtenida durante la consulta.

Figura 20

Respuesta de GET /partidos en la documentación interactiva



Nota. Captura propia de Swagger UI (2026). Comprobación del endpoint que devuelve los registros de votación en formato JSON.

Con el servidor en ejecución, DevTools permitió abrir una conexión ws://127.0.0.1:8000/ws y registrar el mensaje de resultados inicial.

Figura 21

Conexión WebSocket y primer mensaje recibido en DevTools



Nota. Captura propia de la consola (2026). El mensaje recibido contiene tipo resultados y un arreglo de tres partidos.

Figura 22

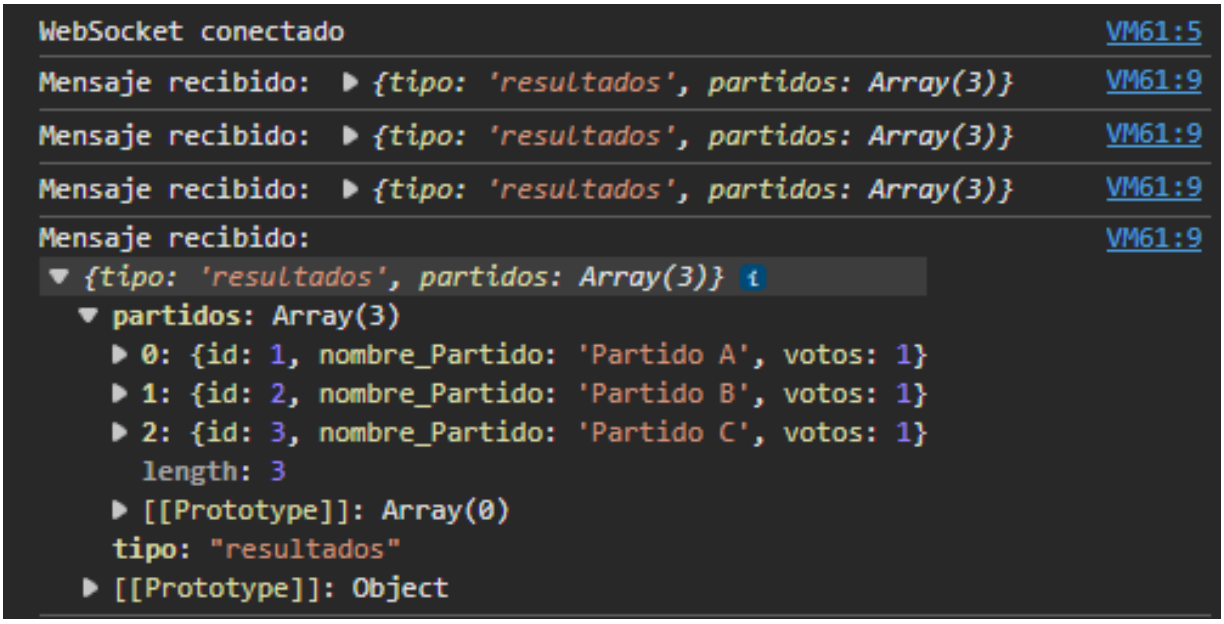
Envío de votos JSON desde la consola del navegador



Nota. Captura propia de DevTools (2026). Se muestran mensajes tipo votar enviados mediante socket.send con el identificador del partido.

Figura 23

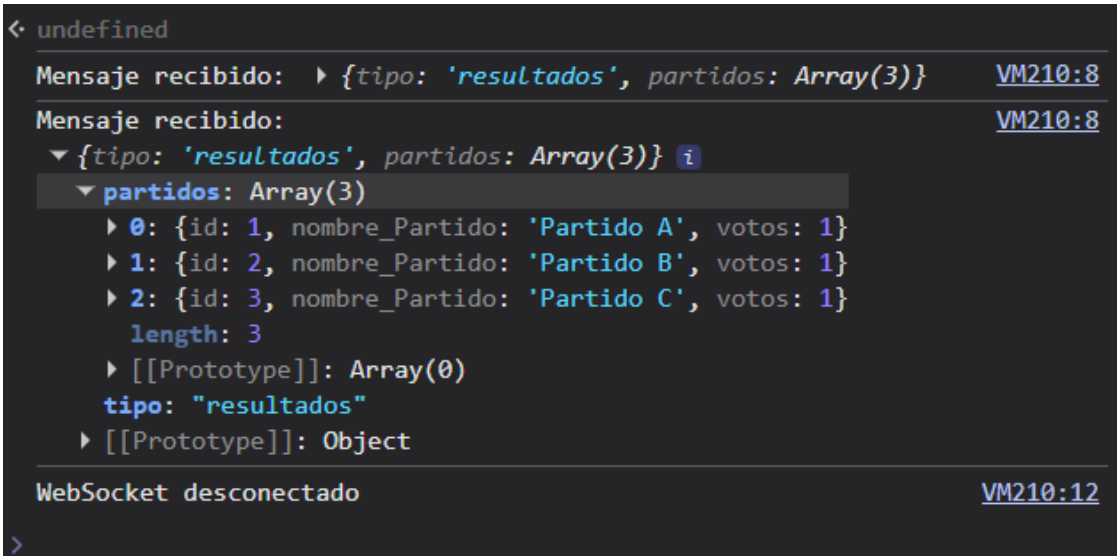
Recepción de mensajes de resultados en la consola del navegador



Nota. Captura propia de DevTools (2026). Se observan múltiples mensajes JSON y el arreglo partidos con sus respectivos contadores.

Figura 24

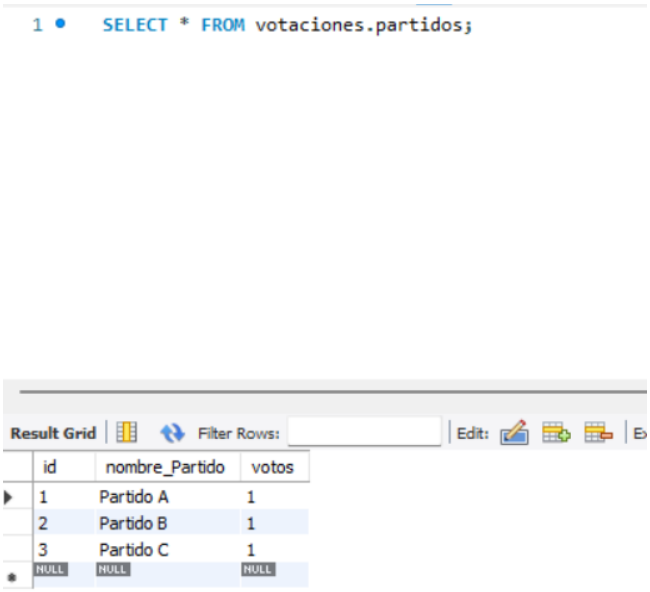
Actualización difundida y valores recibidos en otro navegador



Nota. Captura propia de DevTools (2026). La estructura JSON identifica los tres partidos y evidencia la difusión posterior a un voto.

Figura 25

Estado intermedio de los votos persistidos en MySQL



Nota. Captura propia de MySQL Workbench (2026). En una verificación previa al resultado final los tres partidos presentaban un voto cada uno.

5.2 Broadcast: sincronización en tres navegadores

Se conectaron simultáneamente Chrome, Brave y Edge. Al votar en uno de ellos, los resultados se actualizaron en los otros sin recargar manualmente las páginas. En las Figuras 8, 9 y 10 se observa el mismo estado final: A = 1, B = 2 y C = 1, equivalente a 25 %, 50 % y 25 %, respectivamente. La coincidencia de los resultados y las pruebas operativas verificaron la difusión a los clientes conectados.

Figura 26

Resultados finales observados en Chrome



Nota. Captura propia (2026). Chrome muestra 1, 2 y 1 votos para los partidos A, B y C.

Figura 27

Resultados sincronizados observados en Brave



Nota. Captura propia (2026). Brave recibe los mismos valores del servidor mediante WebSocket.

Figura 28

Resultados sincronizados observados en Edge



Nota. Captura propia (2026). Edge presenta los mismos porcentajes y contadores que los demás navegadores.

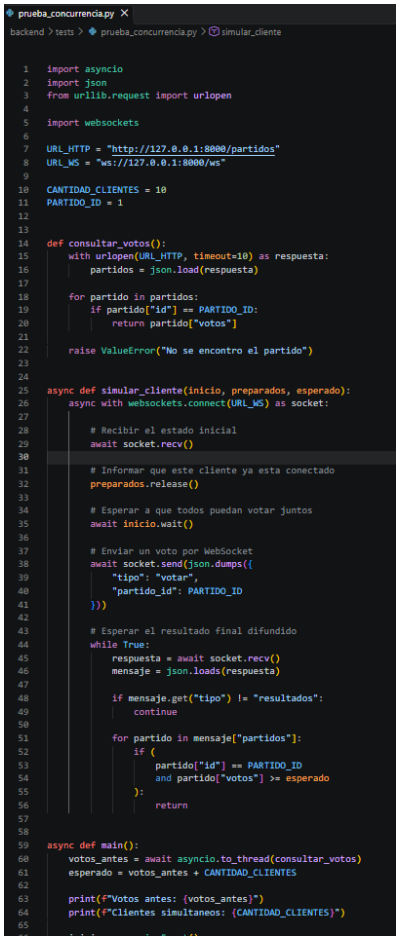
5.3 Verificación de persistencia y concurrencia

La persistencia se verificó mediante una consulta SELECT a la tabla partidos en MySQL Workbench. El registro almacenado coincidió con los contadores mostrados en la interfaz. Posteriormente se ejecutó prueba_concurrencia.py con diez clientes WebSocket que iniciaron sus envíos al mismo tiempo mediante una señal de coordinación.

Para evaluar concurrencia se desarrolló un script en backend/tests/prueba_concurrencia.py. El ensayo establece diez conexiones WebSocket independientes, las sincroniza con asyncio y compara el contador inicial con el final.

Figura 29

Definición de clientes concurrentes y envío del voto de prueba



Nota. Captura propia de prueba_concurrencia.py (2026). Primera parte del script: conexión, espera de inicio y publicación de votos.

Figura 30

Coordinación y verificación del resultado de diez clientes

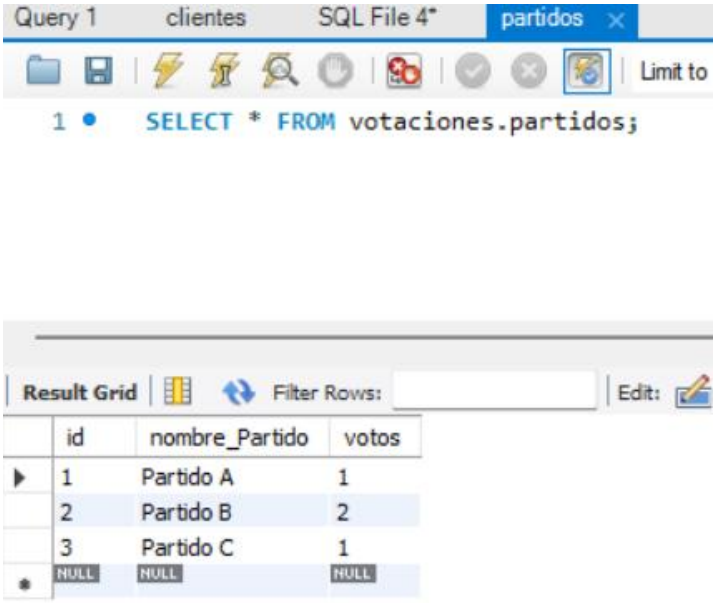


Nota. Captura propia de prueba_concurrencia.py (2026). Segunda parte: creación de tareas y contraste entre votos esperados y registrados.

La salida registrada indica: votos antes = 1; clientes simultáneos = 10; votos después = 11; diferencia = 10. Por tanto, en esta ejecución se recibieron y persistieron los diez incrementos esperados, sin pérdida observada. La prueba no representa una evaluación de cargas prolongadas ni de múltiples instancias del servidor.

Figura 31

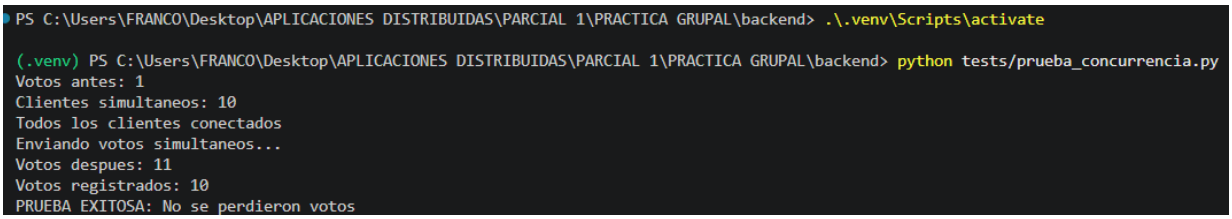
Consulta de los contadores persistentes en MySQL



Nota. Captura propia de MySQL Workbench (2026). La consulta SELECT permite contrastar los valores persistidos con la interfaz.

Figura 32

Resultado de la prueba de concurrencia con diez clientes



Nota. Captura propia de PowerShell (2026). La terminal reporta 1 voto inicial, 11 finales y 10 votos registrados, con resultado satisfactorio.

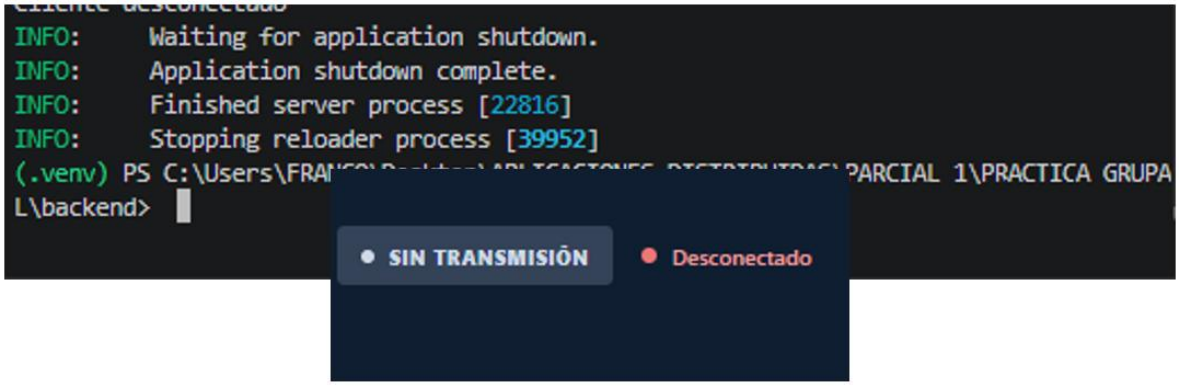
5.4 Detección de desconexión

Se detuvo deliberadamente Uvicorn. El frontend detectó la pérdida de conexión y cambió el indicador EN VIVO / Conectado por SIN TRANSMISIÓN / Desconectado; los botones

quedaron deshabilitados. El cliente de esta versión no vuelve a conectarse por sí solo y requiere una recarga después de reiniciar el backend.

Figura 33

Desconexión controlada de FastAPI y aviso en React



Nota. Composición de capturas propias (2026): detención de Uvicorn y estado de desconexión mostrado por React.

5.5 Registro consolidado de resultados

La Tabla 2 presenta únicamente las validaciones realizadas y sus resultados observados, de acuerdo con el alcance de esta práctica.

Tabla 2

Resumen de las pruebas funcionales realizadas

Validación	Criterio observado	Resultado
Persistencia	El SELECT y la interfaz muestran el mismo conteo.	Satisfactorio
Broadcast	Chrome, Brave y Edge actualizan sin recarga manual.	Satisfactorio
Concurrencia	Diez envíos producen diez incrementos en la prueba.	Satisfactorio
Desconexión	La interfaz muestra estado desconectado al detener Uvicorn.	Satisfactorio

Interpretación. Los ensayos evidencian el funcionamiento de Broadcast, persistencia y concurrencia en un servidor local y bajo los escenarios documentados. No permiten extrapolar el resultado a un entorno electoral real, escalamiento horizontal o disponibilidad continua.

6. Conclusiones

Se alcanzó el objetivo general de construir una aplicación de votación que almacena los resultados en MySQL y los distribuye a varios clientes mediante WebSocket. La interfaz React refleja las actualizaciones sin recurrir a recargas manuales después de cada voto.

La separación entre presentación, aplicación y persistencia facilitó el desarrollo y la comprensión del recorrido de cada operación. ConnectionManager concentró las conexiones activas y la difusión de resultados, mientras que votacion_service.py mantuvo la lógica de actualización separada de la interfaz.

La actualización transaccional del contador y la ejecución coordinada de diez clientes permitieron comprobar que, en la prueba realizada, se registraron exactamente diez incrementos. Este hallazgo respalda la implementación de concurrencia para el escenario evaluado, pero no demuestra su comportamiento bajo cargas indefinidas.

La comparación entre Chrome, Brave y Edge confirmó resultados consistentes en los tres clientes, y la detención del servidor mostró que React reconoce la pérdida de conexión. Estas verificaciones también permitieron identificar como limitación la ausencia de reconexión automática.

7. Recomendaciones

Incorporar un mecanismo de reconexión WebSocket con reintentos y actualización del estado inicial tras interrupciones. Esto permitiría recuperar la comunicación sin depender de una recarga manual del navegador.

Aplicar WSS/HTTPS, restringir los orígenes autorizados, mantener credenciales fuera del código fuente y utilizar un usuario MySQL con privilegios mínimos. Los archivos .env y las contraseñas no deben publicarse en repositorios ni evidencias.

Si el sistema evolucionara más allá de una simulación académica, registrar votos individuales asociados con identificadores de votantes, asegurar unicidad por elección e incorporar trazabilidad y auditoría. La única tabla actual almacena contadores, no historial individual.

Para desplegar varios procesos o servidores, sustituir el listado local de conexiones por una arquitectura de mensajería compartida, por ejemplo un sistema de publicación/suscripción, y repetir pruebas con mediciones de latencia y carga.

8. Referencias

FastAPI. (s. f.). WebSockets. https://fastapi.tiangolo.com/advanced/websockets/

Fette, I., & Melnikov, A. (2011). The WebSocket protocol (RFC 6455). Internet Engineering Task Force. https://doi.org/10.17487/RFC6455

Oracle. (s. f.). UPDATE statement. En MySQL 8.4 reference manual. https://dev.mysql.com/doc/refman/8.4/en/update.html

React. (s. f.). Synchronizing with effects. https://react.dev/learn/synchronizing-with-effects

SQLAlchemy. (s. f.). Working with transactions and the DBAPI. https://docs.sqlalchemy.org/en/20/tutorial/dbapi_transactions.html

Vite. (s. f.). Getting started. https://vite.dev/guide/

9. Anexos

9.1 Anexo A. Estructura del proyecto

```
VotacionTiempoReal/
  backend/
    main.py
    database.py
    connection_manager.py
    votacion_service.py
    tests/prueba_concurrencia.py
  frontend/
    src/App.jsx
    src/App.css
    src/index.css
    src/assets/partidos/partidoA.png
    src/assets/partidos/partidoB.png
    src/assets/partidos/partidoC.png
```

Los archivos .env, .venv y node_modules son propios del entorno local y no se incorporan a las evidencias impresas. Las capturas incluidas en las secciones anteriores documentan los módulos y resultados más relevantes.